Homework 2: Securing the "AutoDrive" Firmware Pipeline

Juan Pablo Bautista, Ryan de la Torre, Daniela Salazar

Scenario: You are a Security Engineer at *AutoDrive*, a company that manages thousands of self-driving cars. You must ensure that when the company sends a software update (firmware) to a car, it is genuine, hasn't been tampered with, and cannot be intercepted by competitors.

Part 1: RSA Key Generation (The Mechanics)

Before a car leaves the factory, it generates its own RSA key pair to communicate securely with AutoDrive HQ.

1. **Manual Calculation:** Assume a simplified RSA setup where the car chooses two small primes: $p = 11$ and $q = 13$.
    - Generate the public and private keys
    - Verify that they can be used for encryption and decryption
2. **Implementation Logic:** Why is it computationally impossible for a hacker to find d if they only know n and e in a real-world 2048-bit system?

Manual Calculation:

calculamos n:

$$n=p\times q=11\times 13=143$$

Calculamos el tociente φ(n):

$$\varphi(n)=(p-1)(q-1)=10\times 12=120$$

ahora debemos de calcular e:

Para eso debe de cumplir:

$$1<e<\varphi(n) \text{ y } \gcd(e,\varphi(n))=1.$$

para esto vamos a escoger el número 7, ya que cumple:

$$\gcd(7\ ,120)=1$$

entonces la clave pública es:

$$(n,e)=(143,7)$$

Ahora vamos a calcular el exponente d (privado):

d es el inverso multiplicativo de *e* módulo φ(n):

$$7d = 1+120k$$

probamos:

- k=1 : 7d=121 → de=121/7 → no entero.
- k=2 : 7d=241 → *d*=121/7 → no entero.
- k=3 : 7d=361 → 361/7=51.571→ no entero.
- k=4 : 7d=481 → *d*=481/7 → 68.17 no entero.
- k=5 : 7d = 601→ 601/7 → 85.857 no entero.
- k= 6 : 7d=721 → *d*=721/7 = 103 → entero.

por tanto d = 103

Comprobamos:

$$7\times1 = 03721 = 6\times120+1$$

la clave privada nos queda: $(n, d) =(143, 103)$

para finalizar comprobamos el cifrado y el descifrado.

elegimos el mensaje m, para este caso usamos m = 20.

Cifrado = m^{e} mod n = $20^{7}\ mod\ 143.$

calculamos:

20^2 = 400 mod 143 = 400−2 × 143 = 400 − 286 = 114

$20^4 = (20^2)^2 = (114)^2 12996$ Dividimos entre 143: 143×90 = 12870

resto : 12996 − 12870 = 126

$20^7 = 20^4 * 20^2 * 20^1 = 126 \times 114 \times 20$ mod 143.

Primero : 126 × 114 = 14364.143 × 100 = 14300, reto 64.

Después: 64 × 20 = 1280.143 × 8 = 1144, resto 1280 ×1144=136.

entonces: C = 136 .

Descifrado: $m' = c^d mod\, n = 136^{103} mod\, 143$.

comprobación:

sabemos que $136 \equiv -7\ (mod\, 143)\ (143 - 7 = 136143 - 7 = 136)$

Entonces: $(-7)^{103} mod\, 143 \ -> c^d$ Cómo 103 es impar $-7^{103} = (-7)^{103}$

por teorema de euler: $7^{\varphi(103)} = 7^{120} \equiv 1$

Podemos reducir el exponente módulo 120: 103 mod 120=103, no se simplifica mucho.

en la teoría: como e · d ≡ 1 mod φ(n), se cumple $m^{ed} \equiv m\ mod(n)$ para mi coprimo de n. En nuestro caso 20 sí es coprimo (20 y 143 no comparten factores), entonces funciona.

Para concluir tenemos que la clave pública (143,7), clave privada (143,103).

Parte 2:

Por qué es imposible encontrar d con solo n y e en RSA real (2048 bits)

En un sistema real, n es el producto de dos primos enormes (cada uno de unos 1024 bits, dando *n* de 2048 bits), además para hallar “d” un atacante necesita conocer p y

q para hallar la función tociente de euler, sin conocer p y q es equivalente a factorizar n en sus factores primos. Sin embargo en la actualidad no existe un algoritmo rápido para factorizar números primos grandes de 2048 bits. Por tanto es imposible encontrar d por la dificultad y los recursos computacionales limitados de la actualidad.

Part 2: Cryptographic Hashes (Integrity Check)

The firmware file is 2GB. We don't want to sign the entire file because it is too slow.

1. **The Avalanche Effect:** If you change exactly **one bit** in the 2GB firmware file and run it through SHA-256 again, how much of the resulting hash should change? Why is this property important for detecting a "Man-in-the-Middle" attack?

El Efecto Avalancha es una propiedad muy deseable de los algoritmos criptográficos, como funciones hash, que indica que un cambio menor en el input, como un cambio en un solo bit, causa un cambio drástico e impredecible en el output (Siddiqui, 2025). Si se cambia exactamente un bit del archivo de firmware de 2 GB, el hash SHA-256 debería cambiar de manera drástica. Debido al Efecto Avalancha, en promedio cambiarán aproximadamente 128 de los 256 bits del hash (Grover's Algorithm Apocalypse by Satoshi is SHA256, 2025). Esta propiedad es importante porque permite detectar cualquier modificación en el archivo. Si un atacante Man in the Middle altera aunque sea una mínima parte del firmware durante la transmisión, el hash calculado por el vehículo será completamente distinto del hash original, por lo que el sistema podrá detectar el ataque y rechazar el archivo.

2. **Collision Resistance:** Explain what a "Collision" is in hashing. If a hacker finds a way to make a malicious "Virus.bin" have the same SHA-256 hash as the "Update.bin," why does this break the entire security of the car?

En hashing, se utilizan funciones hash para generar valores hash. Estos valores se utilizan como índices para las claves dentro de una tabla hash. Una función hash puede devolver el mismo valor hash para dos o más claves. Justamente, una colisión ocurre cuando dos archivos diferentes producen el mismo hash. Matemáticamente , $H(x) = H(y)$, donde $x \neq y$ (Darji, 2024). Si un atacante logra que un archivo Virus.bin tenga el mismo SHA-256

que el archivo legítimo Update.bin, el vehículo no podrá distinguir entre ambos usando solo el hash. Como normalmente se firma el hash del firmware, el atacante podría sustituir el archivo verdadero por el malicioso y el vehículo lo aceptaría como si fuera auténtico. Esto rompe la seguridad del sistema porque destruye la garantía de integridad y permitiría instalar software malicioso en el auto o tener control sobre las funciones del auto.

Part 3: Digital Certificates (The Chain of Trust)

AutoDrive HQ doesn't just send its public key to the car via a text file; it uses a Digital Certificate.

1. **Research Task:** Research the Certificate Signing Request (CSR) process. List the four main pieces of information that AutoDrive HQ must send to a Certificate Authority (CA) to get a signed X.509 certificate.

Para que AutoDrive HQ obtenga un certificado X.509, debe generar un archivo **Certificate Signing Request (CSR)** que vincula su identidad con una clave pública específica y demuestra la posesión de la clave privada asociada.

Las cuatro piezas principales de información que HQ debe enviar a una Autoridad de Certificación (CA) son:

Clave Pública (Public Key):
Es el componente técnico que se incluirá en el certificado y permitirá validar futuras firmas digitales (Kinkelin et al., 2020).

Distinguished Name (DN):
Contiene la identidad del solicitante, incluyendo el Common Name (CN) por ejemplo, el dominio de AutoDrive HQ, así como información de la organización como nombre, unidad organizativa y ubicación geográfica (Kinkelin et al., 2020; Cheng, 1997).

Firma del solicitante (Digital Signature):
El CSR está firmado con la clave privada correspondiente, lo que prueba que el solicitante posee la clave privada asociada a la clave pública enviada (Kinkelin et al., 2020).

Atributos adicionales:
Incluyen información opcional como dirección de correo electrónico o Subject Alternative Names (SANs), utilizados para validación y uso del certificado (Kinkelin et al., 2020).

2. **Validation:** When the car receives the HQ's certificate, it needs to verify it. What specific "Root Certificate" must be pre-installed on the car's computer at the factory for this to work?

Cuando el coche recibe el certificado de la sede central (HQ), debe verificar que sea auténtico y que provenga de una fuente de confianza

Para que este proceso funcione, debe estar preinstalado en la computadora del coche el certificado raíz (Root Certificate) de la Autoridad de Certificación (CA) que se encuentra en la cima de la cadena de confianza (BSI, 2026).

Lógica de validación:
En una Infraestructura de Clave Pública (PKI), el certificado de AutoDrive HQ normalmente es firmado por una Autoridad de Certificación intermedia, que a su vez está firmada por la CA raíz.
El certificado raíz actúa como el “ancla de confianza”, ya que su clave pública es distribuida de forma segura en los dispositivos durante su fabricación (BSI, 2026).

El proceso de verificación consiste en validar toda la cadena de confianza:

Certificado de AutoDrive HQ, Certificado de la CA intermedia y Certificado raíz (preinstalado)

Si el coche no posee este certificado raíz, no podrá validar la cadena de confianza y rechazará cualquier actualización como potencialmente maliciosa (BSI, 2026).

Part 4: Digital Signatures (Authentication)

To prove the update is official, AutoDrive HQ "signs" the update.

1. **The Workflow:** Describe the process of creating the signature.
    - What is hashed?
    - Which key (HQ Public or HQ Private) is used to encrypt the hash?

Se hashea el contenido del firmware (el archivo Update.bin). El hash es una "huella digital" de 256 bits (si usamos SHA-256). Es decir, $hash = SHA256(Update.bin)$.

Luego, AutoDrive HQ genera la firma digital utilizando su clave privada sobre ese hash. Después envía al vehículo el firmware y la firma. El vehículo calcula nuevamente el hash del archivo recibido y verifica la firma usando la clave pública de HQ. Si el hash calculado coincide con el hash validado por la firma, entonces el vehículo sabe que el archivo no fue alterado y que realmente fue autorizado por AutoDrive HQ.

2. **Non-Repudiation:** If a faulty update causes a car to crash, AutoDrive might try to claim they never sent that specific update. How does the use of a **Digital Signature** provide legal "Non-repudiation" in this case?

La firma digital proporciona no repudio porque solo AutoDrive HQ debe tener acceso a su clave privada. Si una versión específica del firmware fue firmada con esa clave, entonces HQ no puede negar fácilmente haber autorizado esa actualización. En caso de un incidente, la firma digital puede verificarse con la clave pública de HQ y sirve como evidencia criptográfica de que esa versión concreta fue aprobada por la empresa.

Part 5: System Design Challenge

The Task: Design a "Secure Update Protocol" for the car. Draw a flow diagram or provide a numbered list showing the interaction between **AutoDrive HQ** and the **Vehicle**.

Your design must use a **Hybrid System** (Asymmetric for keys, Symmetric for the 2GB file) and satisfy these five requirements:

Requirement	**How your design achieves it**
1. Secure Key Exchange	How does the car get the AES "Session Key" from HQ?
2. Confidentiality	How do you ensure a competitor cannot "sniff" the firmware code?
3. Integrity	How does the car know the file wasn't corrupted during the download?
4. Authentication	How does the car know the update is from HQ and not a "Spoofed" server?

5. Non-Repudiation	How do we prove HQ is the one who authorized this specific version?

Este protocolo combina criptografía asimétrica (RSA) para la gestión de claves y firmas y con criptografía simétrica (AES) para procesar de manera eficiente el archivo de 2GB.

Tenemos en cuenta que hay varias cosas que deben estar instaladas previamente. El vehículo debe tener la clave privada propia, la clave pública propia, y el Certificado Raíz de la CA. AutoDrive HQ debe tener el par de claves (pública y privada), el Certificado X.509 válido y acceso a la clave pública de cada vehículo.

A continuación se muestra el flujo o lista numerada del protocolo paso a paso:

En AutoDrive HQ

1. AutoDrive HQ debe calcular el hash del firmware $H = SHA256(Update.bin)$. Esto es el hashing
2. HQ firma ese hash con su clave privada $firma = Sign_{PrivadaHQ}(H)$. Aquí se crea la firma digital.
3. HQ genera una clave aleatoria AES-256 (K_{AES}). Esta es una clave para la sesión
4. Se cifran los datos, es decir, el firmware de 2GB. $C = AESEncrypt(K_{AES}, Update.bin)$
5. Se protege la clave AES usando la clave pública del carro. Este es el cifrado de la clave. $E = RSAEncrypt\ (K_{públicaVehículo}, K_{AES})$

Transmisión y verificación en el vehículo

6. HQ envía el paquete con el firmware cifrado (C), la clave (E), la firma, y el certificado HQ.
7. El vehículo tiene que verificar que el certificado HQ sea auténtico utilizando el Certificado Raíz de confianza
8. El vehículo debe usar su clave privada para obtener la clave AES, es decir, recupera la clave. $K_{AES} = RSADecrypt\ (K_{privadaVehículo}, E)$
9. El vehículo, con K_{AES}, debe descifrar el contenido para obtener el firmware original. $Update.bin = AESDecrypt\ (K_{AES}, C)$

10. El vehículo debe calcular su propio hash (H_1). Verifica que la firma de HQ sea auténtica utilizando la clave pública de HQ y obtiene el hash original H_2. H_1 y H_2 deben ser iguales para confirmar que el firmware es auténtico y no tuvo ninguna modificación.

El protocolo de actualización segura de Autodrive utiliza un sistema híbrido. Primero, HQ calcula el hash SHA-256 del firmware y firma ese hash con su clave privada, generando una firma digital. Luego genera una clave de sesión AES para cifrar el archivo de firmware de 2 GB, ya que el cifrado simétrico es más eficiente para archivos grandes. Después, HQ cifra la clave AES con la clave pública del vehículo, de modo que solo ese vehículo pueda recuperarla con su clave privada.

Luego, HQ manda al vehículo el firmware cifrado, la clave AES cifrada, la firma digital y su certificado digital. El vehículo primero valida el certificado de HQ usando el certificado raíz preinstalado. Si el certificado es válido, descifra la clave AES con su clave privada y luego usa esa clave para descifrar el firmware. Finalmente, calcula el hash del firmware recibido y verifica la firma digital usando la clave pública de HQ. Si el hash calculado coincide con el hash autenticado por la firma, el vehículo acepta e instala la actualización.

Requerimientos

1. **Secure key exchange:** HQ cifra la clave de la sesión AES con la clave pública del vehículo. Solo el vehículo puede recuperarla con su clave privada única
2. **Confidentiality:** el firmware de 2 GB se cifra con AES. Un atacante que intercepte el tráfico sólo verá datos ilegibles sin la clave de sesión
3. **Integrity:** el vehículo compara el hash calculado localmente con el hash firmado por HQ. Cualquier cambio, incluso si fue en un solo bit, será detectado
4. **Authentication:** el vehículo valida el certificado digital de HQ y la firma. Esto asegura que la actualización viene del servidor oficial y no de un impostor
5. **Non-repudiation:** la firma digital vincula criptográficamente a HQ con esa versión del firmware. HQ no puede negar haber autorizado esa actualización específica.

Referencias

BSI - Federal Office for Information Security. (2026). *Cryptographic mechanisms: Recommendations and key lengths (Version 2026-01).* www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.pdf?__blob=publicationFile&v=10

Cheng, E. (1997). *An introduction to digital signatures.* SAS-Space. www.sas-space.sas.ac.uk/3629/1/1597-1983-1-SM.pdf

Darji, A. Diciembre de 2024. Understanding Hash Collisions in HashTables: Causes, Impact, and Solutions. Medium. https://medium.com/@arjun.darji/understanding-hash-collisions-in-hashtables-causes-impact-and-solutions-d14a6745f4d4

Grover's Algorithm Apocalypse by Satoshi is SHA256. (3 de noviembre de 2025). Cryptography Devel-2ba7ad4ab2a3

Kinkelin, H., von Seck, R., Rudolf, C., & Carle, G. (2020). *Hardening X.509 certificate issuance using distributed ledger technology.* arXiv. www.arxiv.org/pdf/2004.07063

Siddiqui, L. (27 de octubre de 2025). *Hashing in Cryptography explained: how it works, algorithms, and Real-World uses | Splunk*. Splunk. https://www.splunk.com/en_us/blog/learn/hashing-cryptography.htmlopment: Understanding the Avalanche Effect in Hash Functions. Medium. https://medium.com/@iuecsoranetwork/cryptography-development-understanding-the-avalanche-effect-in-hash-functions